Understanding SIEM

A hands-on guide built around a simple, console-based, Docker-powered SIEM you can run yourself

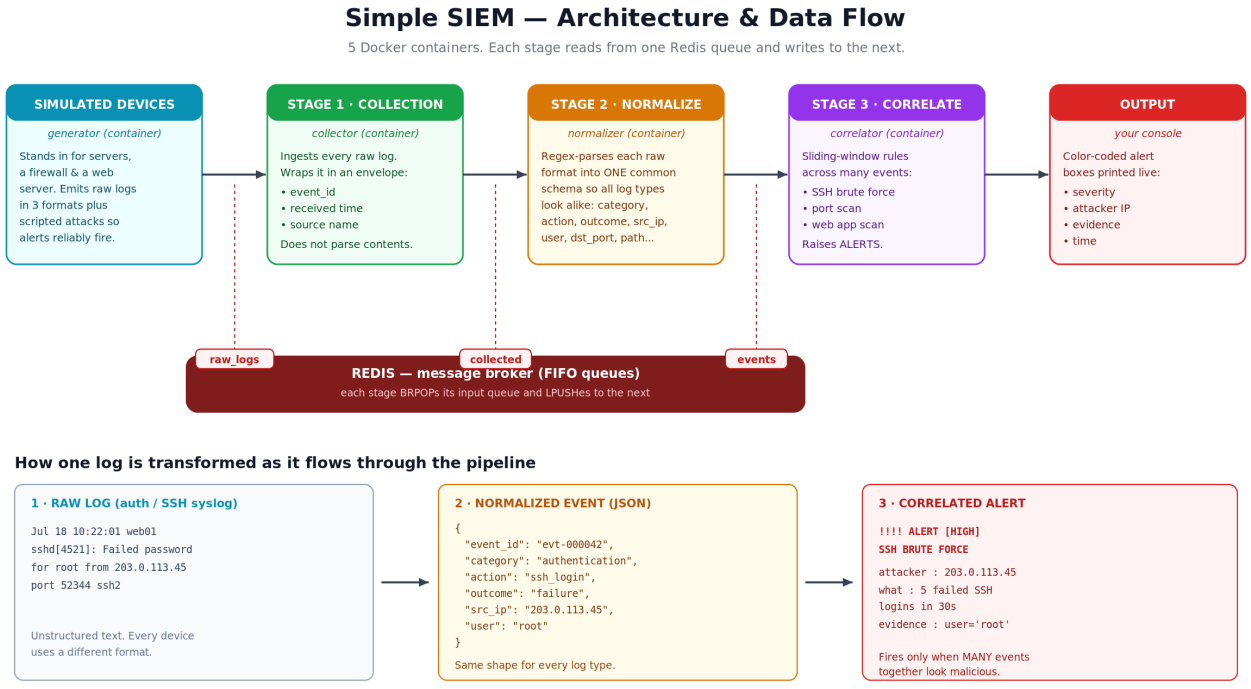

Figure 1 — End-to-end architecture and data flow of the Simple SIEM.

1. What is a SIEM?

A **SIEM** — Security Information and Event Management — is the system a security team uses to watch everything happening across a network in one place. Every server, firewall, router, application, and cloud service constantly produces *logs*: small text records of who did what and when. On their own these logs are scattered across hundreds of machines, written in dozens of incompatible formats, and far too numerous for anyone to read. A SIEM exists to solve that problem.

At its core, a SIEM does three things, and this project is organised around exactly those three stages:

- **Collection** — gather logs from every source into one pipeline.
- **Normalization** — translate every log format into one common structure so they can be compared.
- **Correlation & Alerting** — analyse many events together to detect attacks, and raise an alert when something looks malicious.

A real SIEM (Splunk, Elastic Security, Microsoft Sentinel, Wazuh, and others) adds storage, search, dashboards, and long-term retention on top. But the three stages above are the beating heart of every one of them. Understanding them is the goal of this project.

Why not just read the logs directly?

Imagine a single failed SSH login on one server. Harmless — people mistype passwords. Now imagine ten failed logins from the same foreign IP address in thirty seconds, immediately followed by a success. That is almost certainly an attacker who just guessed a password. No human is watching every server at 3 a.m., and the meaningful signal is spread across many individual log lines. A SIEM connects those dots automatically. That connecting-the-dots is called *correlation*, and it is the reason SIEMs exist.

2. The three stages, and how this system demonstrates each one

Stage 1 — Collection

The concept: pull raw logs in from every source and get them into a single pipeline, without yet trying to understand their contents. This is the SIEM's front door.

In this system: the `collector` container reads every raw log off a queue and wraps each one in a common 'envelope' — a unique `event_id`, the time it was received, and which source it came from — then passes it on. It deliberately does not parse the message yet, which keeps the stage simple and its single job obvious.

Stage 2 — Normalization

The concept: different devices describe the same kind of event in completely different text. Normalization rewrites all of them into one shared schema, so that an SSH log, a firewall log, and a web-server log can be counted and compared using the same fields. This is the single most important

idea in a SIEM.

In this system: the `normalizer` container applies one regular expression per source format and produces a uniform JSON event with fields such as `category`, `action`, `outcome`, `src_ip`, `user`, `dst_port`, and `path`. Three unrecognisable raw lines become three events with identical shape:

```
RAW  (auth)     Jul 18 10:22:01 web01 sshd[4521]: Failed password for
                root from 203.0.113.45 port 52344 ssh2
RAW  (firewall) FW: DENY TCP 198.51.100.23:44123 -> 10.0.0.5:3389 len=52
RAW  (web)      192.0.2.10 - - [.../...] "GET /admin.php HTTP/1.1" 404 ...
                          |
                          v   after normalization
{ "category": "authentication", "action": "ssh_login",
  "outcome": "failure", "src_ip": "203.0.113.45", "user": "root" }
```

Stage 3 — Correlation & Alerting

The concept: examine many normalized events over a window of time and decide when a *pattern* is an attack. One event rarely matters; the relationship between events does.

In this system: the `correlator` container keeps a short, per-IP sliding-window history in memory and runs a small set of detection rules on every event. When a rule's threshold is crossed it prints a clearly formatted, colour-coded alert box to the console. The built-in rules are:

Rule	Fires when	Severity
SSH brute force	≥ 5 failed SSH logins from one IP within 30s	HIGH
Brute-force success	a successful login right after those failures	CRITICAL
Port scan	one IP blocked on ≥ 10 different ports within 30s	MEDIUM
Web application scan	≥ 4 suspicious web requests (sqlmap, /admin, ../, SQLi)	HIGH

All thresholds are plain constants at the top of `correlator.py` — students can change them, re-run, and watch how detection sensitivity (and false positives) change.

3. How the system is built

The whole thing runs as five Docker containers started with a single command. Four of them are short Python programs that share one Docker image; the fifth is Redis, which acts as the conveyor belt between stages.

- **generator** — simulates the network's devices. It emits fake auth, firewall, and web logs as background 'noise', and every so often launches a scripted attack so alerts reliably appear.
- **collector** — Stage 1 (collection).
- **normalizer** — Stage 2 (normalization).
- **correlator** — Stage 3 (correlation & alerting).

- **redis** — an in-memory data store used here as simple FIFO queues that connect the stages.

The stages never call each other directly. Each one only reads from its input queue and writes to the next, using two Redis operations:

```
while True:
    msg = pop(from_my_input_queue)     # BRPOP: sleep until work arrives
    result = do_my_job(msg)            # collect / normalize / correlate
    push(to_the_next_queue, result)    # LPUSH: hand off to the next stage
```

The three queues carry data along the pipeline:

```
generator  --[ raw_logs  ]-->  collector
collector  --[ collected ]-->  normalizer
normalizer --[ events    ]-->  correlator  --> ALERTS on your screen
```

Because the stages are decoupled through queues, each is an independent container that could be scaled, restarted, or replaced on its own. This is exactly how production SIEM pipelines separate ingestion, parsing, and detection — just with heavier tools (Kafka, Logstash, and so on) in place of Redis and Python.

4. How to run and use it

Prerequisites

You only need **Docker Desktop** (or Docker Engine with the Compose plugin) installed. No Python setup is required on your machine — everything runs inside the containers.

Start everything with one command

```
cd simple-siem
docker compose up --build
```

Docker builds one small image, then starts all five containers. Within a minute or two you will see colour-coded logs from every stage interleaved in your terminal, and the correlator will begin printing red alert boxes when the generator launches an attack.

Focus on a single stage

The combined stream is busy on purpose — it shows the whole pipeline at once. To watch just one stage, open a second terminal and follow that container's logs:

```
docker compose logs -f correlator      # just the alerts
docker compose logs -f normalizer      # watch raw logs become JSON
docker compose logs -f generator       # see the simulated attacks start
```

Stop and clean up

```
# press Ctrl+C in the terminal running compose, then:
docker compose down
```

What a firing alert looks like

When a rule trips, the correlator prints a block like this:

```
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
  ALERT #1  [HIGH]  SSH BRUTE FORCE
  attacker : 203.0.113.45
  what     : 5 failed SSH logins in 30s
  evidence : target user='root'
  time     : 2026-07-18 10:22:03
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

Read it top to bottom: the severity and rule name tell you *what* was detected, the attacker line tells you *who*, and the evidence line tells you *why* the SIEM believes it is an attack.

5. Ideas for students to extend it

- Add a brand-new log source (DNS or VPN logs): write a format in the generator, a matching regex in the normalizer, and a rule in the correlator.
- Add a rule that alerts when the same user logs in from two different countries in a short window (impossible travel).
- Write alerts to a file or a dedicated Redis queue instead of the screen, then build a tiny dashboard that reads them.
- Lower the thresholds and watch false positives appear — the real tuning problem every security operations centre faces.

This is a learning tool, not a production security product. All logs are synthetic and the detection logic is intentionally simplified so that the core ideas stay visible.